

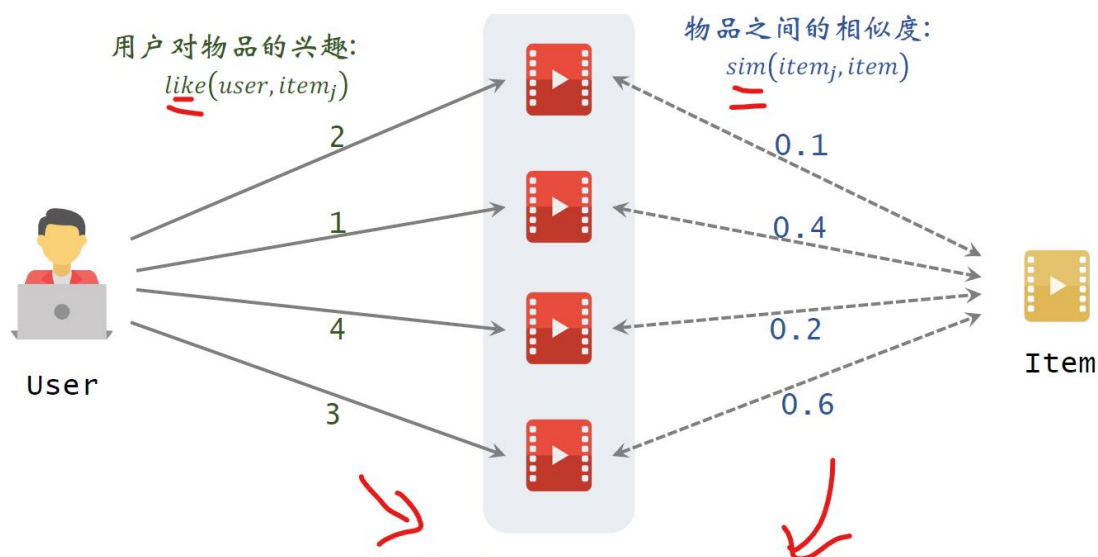
总结

漏斗

- **召回**：用多条通道，取回几千篇笔记。
- **粗排**：用小规模神经网络，给几千篇笔记打分，选出分数最高的几百篇。
- **精排**：用大规模神经网络，给几百篇笔记打分。
- **重排**：做多样性抽样、规则打散、插入广告和运营笔记。

CSDN @冰露可乐

召回 1：基于物品的协同过滤（ItemCF）



预估用户对候选物品的兴趣： $\sum_j \text{like}(\text{user}, \text{item}_j) \times \text{sim}(\text{item}_j, \text{item})$

CSDN @冰露可乐

黄色物品为新物品，红色物品为用户历史交互过的物品。我们首先要获得用户与历史交互物品的喜好值，可以量化用户对物品的兴趣来获得，比如点击、点赞、收藏、转发的四种行为各算一分。然后要获得新物品与历史物品的相似度。新物品的推荐分数即求积（用户对某物品的喜好值*新旧物品的相似性），根据该分数决定是否推荐。

物品间的相似度怎么计算：基本想法是这样的，两个物品的受众重合度越高，两个物品就越相似。即如果喜欢物品 i_1 的用户，和喜欢物品 i_2 的用户高度重合，那么这两个物品就相似。

原来是交集，现在是对那俩物品的喜欢程度

计算物品相似度

- 喜欢物品 i_1 的用户记作集合 $\mathcal{W}_1$ 。
- 喜欢物品 i_2 的用户记作集合 $\mathcal{W}_2$ 。
- 定义交集 $\mathcal{V} = \mathcal{W}_1 \cap \mathcal{W}_2$ 。
- 两个物品的相似度：

$$sim(i_1, i_2) = \frac{\sum_{v \in \mathcal{V}} like(v, i_1) \cdot like(v, i_2)}{\sqrt{\sum_{u_1 \in \mathcal{W}_1} like^2(u_1, i_1)} \cdot \sqrt{\sum_{u_2 \in \mathcal{W}_2} like^2(u_2, i_2)}}$$

余弦相似度 (cosine similarity)

GSDN @冰露可乐

然后取连加，连加是关于用户小v取的，

用户小v同时喜欢两个物品，如果兴趣分数的取值是零或者一，那么分子就是同时喜欢两个物品的人数，也就是集合V的大小，

公式的分母是两个根号的乘积，第一项是用户对物品的兴趣分数。

分母是关于所有用户，求连加，然后开根号第二项类似用户对物品l的兴趣分数。

这个公式计算出的数值介于零到一之间表示两个物品的相似度，

其实这个公式就是余弦相似度cos similarity

把一个物品表示为一个向量，向量每个元素对应一个用户元素的值就是用户对物品的兴趣，

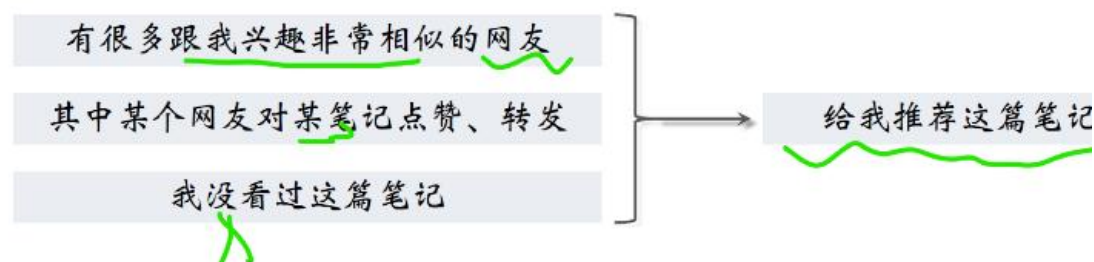
Like(u1,i1)为喜欢物品 i1 的某个用户 u1 对其的 like 程度。本质为余弦相似度来评判物体间的相似度。

召回 2：基于用户的协同过滤（UserCF）

当时我讲Item cf是基于物品之间的相似性做推荐，

而User cf是基于用户之间的相似性做推荐，

UserCF的原理

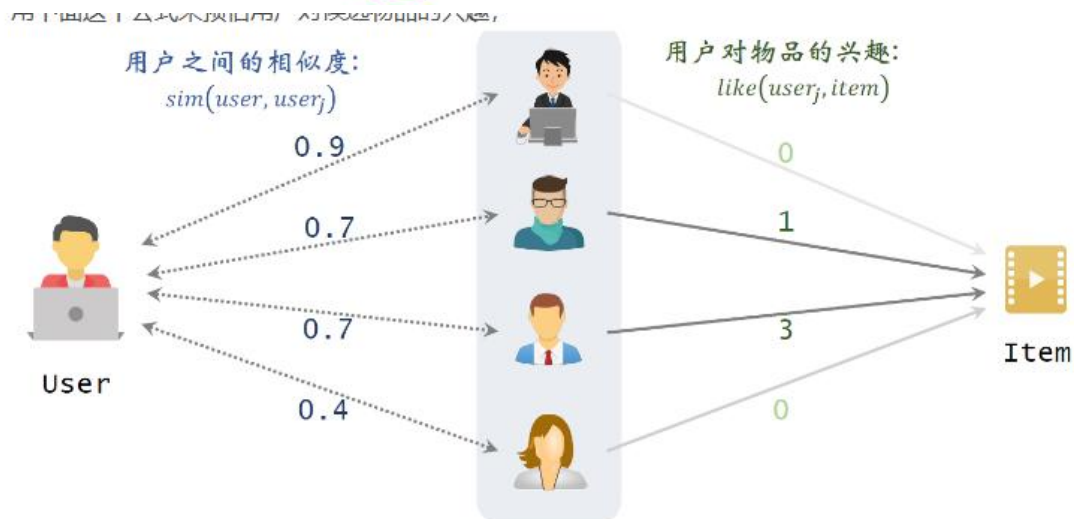


苦生解释usercf的原理 作为小红书的用户 我在小红书的占主 占赞 收藏 转发可以体现出我的

推荐系统如何找到跟我兴趣非常相似的网友呢？

- 方法一：点击、点赞、收藏、转发的笔记有很大的重合。
- 方法二：关注的作者有很大的重合。

CSDN @冰露可乐



预估用户对候选物品的兴趣： $\sum_j \underbrace{sim(user, user_j)} \times \underbrace{like(user_j, item)}$

黄色为推荐的物品，被推荐用户为左侧一个，他没有与黄色物品交互过。中间的人为与 USER 相似的用户，他们和物品交互过。

因此，求物品相似度，看喜欢这些物品的用户的状况

反之，求用户相似度，看用户喜欢过的物品的状况

计算用户相似度

- 用户 u_1 喜欢的物品记作集合 J_1 。
- 用户 u_2 喜欢的物品记作集合 J_2 。
- 定义交集 $I = J_1 \cap J_2$ 。
- 两个用户的相似度：

$$sim(u_1, u_2) = \frac{|I|}{\sqrt{|J_1| \cdot |J_2|}}$$

但上面求 sim 的公式有个不足之处：公式同等对待热门和冷门的物品，这是不对的。拿书籍推荐举个例子，哈利波特是非常热门的物品，不论是大学教授还是中学生都喜欢看哈利波特。既然所有人都喜欢看哈利波特，那么哈利波特对于计算用户相似度是没有价值的。越热门的物品越无法反映出用户独特的兴趣，对计算相似度就没有用。反过来，重合的物品越冷门越能反映出用户的兴趣。

为了更好的计算用户兴趣的相似度，我们需要降低热门物品的权重，改为下面的形式：

- 用户 u_1 喜欢的物品记作集合 J_1 。
- 用户 u_2 喜欢的物品记作集合 J_2 。
- 定义交集 $I = J_1 \cap J_2$ 。
- 两个用户的相似度：

$$sim(u_1, u_2) = \frac{\sum_{l \in I} \frac{1}{\log(1 + n_l)}}{\sqrt{|J_1| \cdot |J_2|}}.$$

n_l ：喜欢物品 l 的用户数量，反映物品的热门程度

就是标为红色的那一项。

公式中的其他部分没有变，1除以 $\log(1+n_l)$ 表示物品的权重，

n_l 是喜欢物品 l 的用户数量，反映物品的热门程度。

喜欢哈利波特的人数很多， n_l 自然就会很大，物品越热门，

n_l 越大，1除以 $\log(1+n_l)$ 就越小，也就是说物品的权重越小。

这样一来，哈利波特对相思路的贡献就会很小，

同理，而冷门物品的贡献会比较大。

小结

✓ UserCF 的基本思想：

- 如果用户 $user_1$ 跟用户 $user_2$ 相似，而且 $user_2$ 喜欢某物品，
- 那么用户 $user_1$ 也很可能喜欢该物品。
- 预估用户 $user$ 对候选物品 $item$ 的兴趣：rate

$$\sum_j sim(user, user_j) \times like(user_j, item).$$

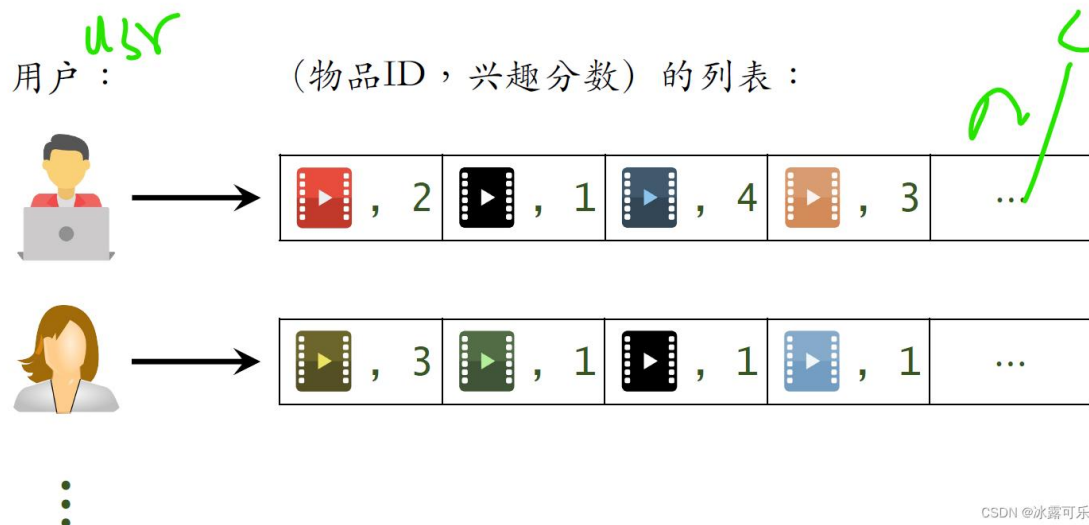
• 计算两个用户的相似度：

- 把每个用户表示为一个稀疏向量，向量每个元素对应一个物品。
- 相似度 sim 就是两个向量夹角的余弦。

CSDN @冰露可乐

使用前，要离线建立两个索引：

“用户→物品”的索引



“用户→用户”的索引



流程：

用户刷小红书之后，系统要给这位用户做推荐，知道用户的 ID，利用用户到用户的索引找到最相似的 k 个用户，取回列表中记录的 top k 的用户的 ID 和相似度。

接下来，我们要利用用户到物品的索引，找到每个用户近期感兴趣的物品，知道这个用户的 ID，取回他近期感兴趣的物品。

我们把这些物品叫做 last n ，

这些就是用户最近感兴趣的物品的 ID 和兴趣分数。

在这个例子中， n 等于 2

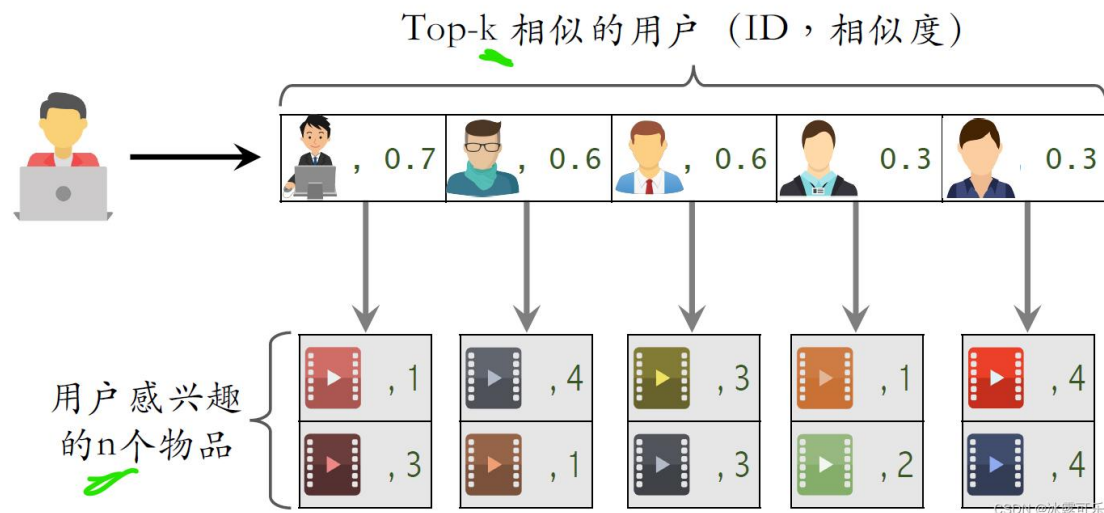
用同样的方法，根据用户到物品的索引，找到其他的每个用户近期感兴趣的物品，一共用了 k 相似用户，每个用户最多有 N 个感兴趣的物品，

因此会取回 N 乘以 k 个物品，

用前面介绍过的公式计算用户对这 N 乘以 k 物品的兴趣分数 rate。

根据算出的兴趣分数 rate 做排序，返回排在前 100 的物品，
也就是说，从这 N 乘以 K 物品中选出 100 个作为 user cf 的召回结果。

线上做召回



用到上面列表中用户之间的相似度，
用到下面列表中用户对物品的兴趣分数，
把上面的相似度跟下面的兴趣分数相乘，得到预估的兴趣分数 rate。
如果取回的物品有重复的就做，去重把分数加起来

在工业界的推荐系统中，**Item CF** 和 **user CF** 都是主要的召回通道
为了在线上做非常快速的召回，需要离线维护两个索引，
一个索引是从用户到物品列表，记录了每个用户近期交互过的 N 个物品，
另一个索引是从用户到用户列表，记录了每个用户像素最高的 K 个用户
在线上做召回的时候要用到这两个索引，每次取回 N 乘以 K 个物品。

召回前置内容：

1. 离散特征处理

离散特征的处理分两步，**第一步是建立字典**，把类别映射成序号。

第二步是做向量化，把序号映射成向量。

one hot 编码是一种常见的方法，把序号映射成高维的稀疏向量。

比如有 200 个国家，每个国家被映射成一个 200 维的向量，序号对应的位置的元素是一，其他位置的元素都是零。

更高级的方法是 **embedding**，把序号映射成低维稠密向量，比如处理国籍特征，每个国家被映射成一个八维的稠密向量。

离散特征处理

1. 建立字典：把类别映射成序号。

- 中国 $\rightarrow$ 1
- 美国 $\rightarrow$ 2
- 印度 $\rightarrow$ 3

2. 向量化：把序号映射成向量。

- One-hot编码：把序号映射成高维稀疏向量。
- Embedding：把序号映射成低维稠密向量。

CSDN @冰露可乐

One-Hot 编码：

例2：国籍特征

- 国籍：中国、美国、印度等 200 种类别。
- 字典：中国 $\rightarrow$ 1，美国 $\rightarrow$ 2，印度 $\rightarrow$ 3， ...
- One-hot编码：用 200 维稀疏向量表示国籍。
 - 未知 $\rightarrow$ 0 $\rightarrow$ [0, 0, 0, 0, ..., 0]
 - 中国 $\rightarrow$ 1 $\rightarrow$ [1, 0, 0, 0, ..., 0]
 - 美国 $\rightarrow$ 2 $\rightarrow$ [0, 1, 0, 0, ..., 0]
 - 印度 $\rightarrow$ 3 $\rightarrow$ [0, 0, 1, 0, ..., 0]

CSDN @冰露可乐

One-Hot编码的局限

- 例1：自然语言处理中，对**单词**做编码。
 - 英文有几万个常见单词。
 - 那么one-hot向量的维度是**几万**。
- 例2：推荐系统中，对**物品ID**做编码。
 - 小红书有几亿篇笔记。
 - 那么one-hot向量的维度是**几亿**。

类别数量太大时，通常不用 one-hot 编码。

CSDN @冰露可乐

embedding 特征嵌入

把每个类别映射成一个低维的稠密向量。

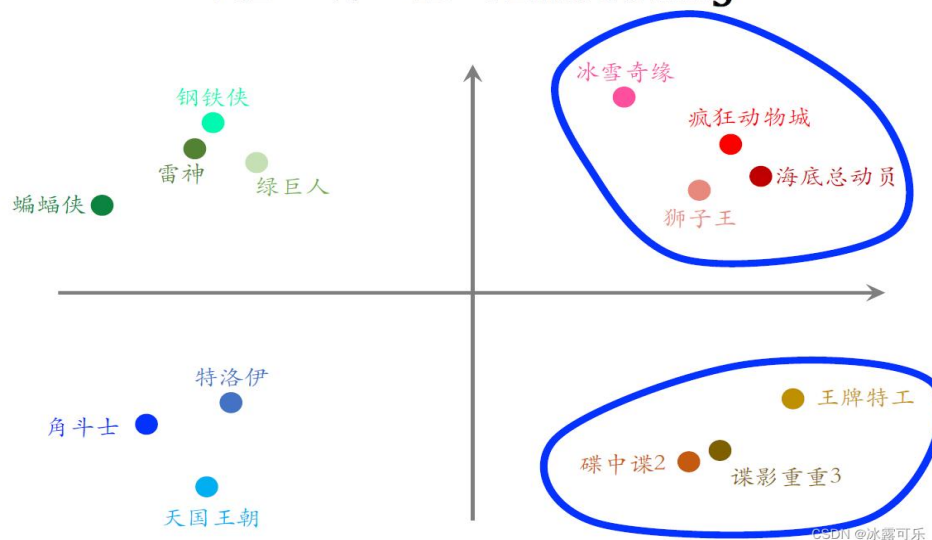
例子是国籍，字典里有中国、美国、印度、日本、德国等 200 个国家。

字典把每个国家映射成一个序号，比如中国是一，美国是二，印度是三一共有 200 个序号。

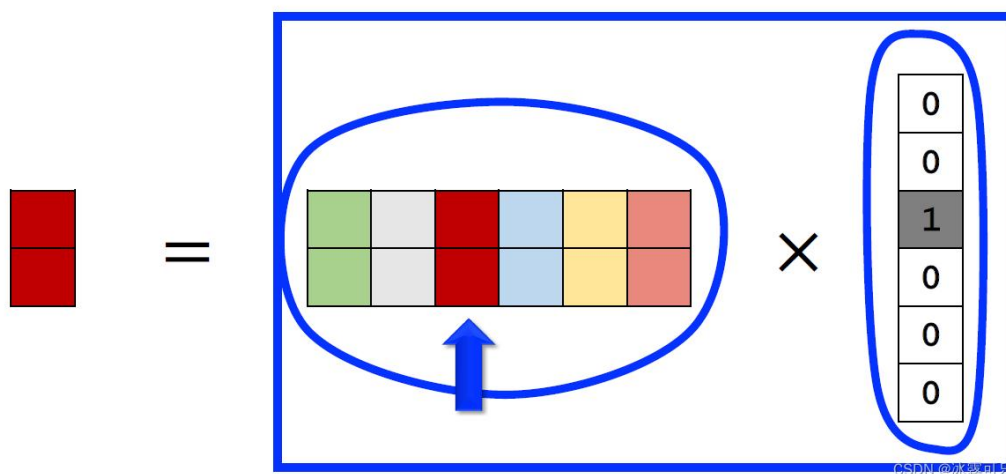
embedding 把每个序号映射成一个向量，这些向量都是低维向量，比如向量大小都是四乘以一个的向量就是对一个国家的表示，未知国籍就用全零向量表示。

在训练神经网络的时候会自动做反向传播，学习 embedding 的参数。假如美国 ID = 2，我们只需要在用到时去 embedding 矩阵中提取第二列的向量即可。

例2：物品ID的Embedding



$$\text{Embedding} = \text{参数矩阵} \times \text{One-Hot向量}$$



embedding 其实就是把 one hot 的向量乘到一个参数矩阵 W 上得到的精致向量。

举个例子，右边是一个 one hot 的向量，它的第三个元素是一，其余元素全都是零，中间是 embedding 乘的参数矩阵 W，把矩阵 W 和 one hot 向量相乘。由于 one hot 向量只有第三个元素非零，所以矩阵向量乘法其实就是取出矩阵 W 的第三列，把第三列作为输出，输出的向量就是参数矩阵×one hot 向量。

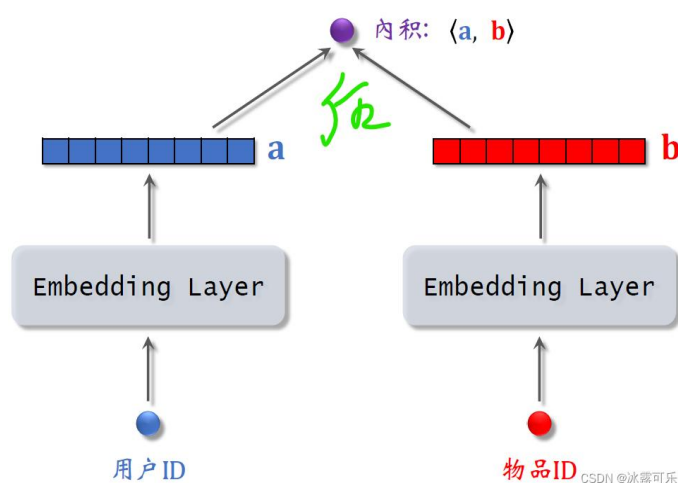
怎么计算最近邻，即怎么找到距离最近的两个向量？

可以用欧式距离最小、向量内积最大、向量余弦相似度最大

普通找相似的两个商品，需要把所有商品遍历一遍查找，所以有了加速最近查找的算法，在做线上服务之前，先对数据做预处理，把数据划分成很多区域。划分区域之后，每个区域都用一个单位向量来表示，用户向量到来时，先和该总结性的单位向量做计算，找到某一个单位向量，然后在和该区域的所有向量进行计算。

双塔模型：

矩阵补充模型



上图是矩阵补充的模型，将用户 ID 和物品 ID 进行 EMBEDDING 后计算内积，判断是否喜好。但这种方法利用用户和商品信息太少了。双塔模型增加了利用的内容。

用户的特征：我们知道用户 ID 还能从用户填写的资料和用户行为中获取很多特征，包括离散特征和连续特征。对于每个离散特征，用单独一个 embedding 层得到一个向量，比如用户所在城市，用一个 embedding 层。对于性别这样类别数量很少的离散特征，直接用 one hot 编码就行，可以不做 embedding。

用户还有很多连续特征，比如年龄、活跃程度、消费金额等等。**连续特征的定义为：**数字间有数学意义，比如年龄可以比较、求平均、求差值。不同类型的连续特征有不同的处理方法，最简单的是做归一化，让特征均值是零，标准差是一。有些长尾分布的连续特征需要特殊处理，比如取 log，比如做分筒。

基本流程：分别归一化 → 拼接

用户特征：

连续特征1：年龄 = 25
连续特征2：收入 = 8000
连续特征3：活跃天数 = 30

步骤1：分别归一化

年龄归一化（范围 0-100）：
 $25 \rightarrow (25-0)/100 = 0.25$

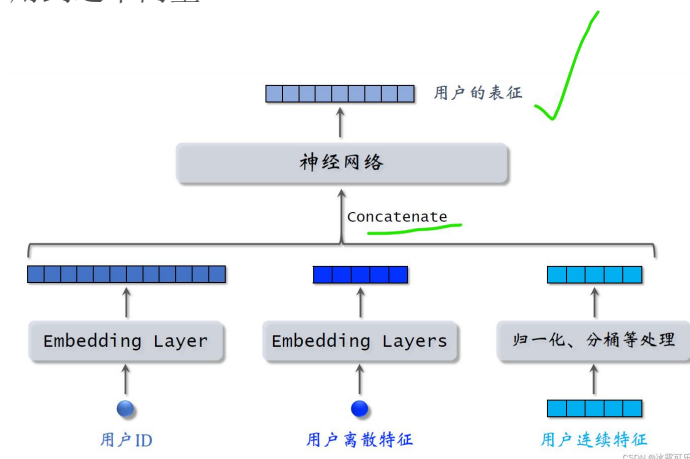
收入归一化（范围 0-50000）：
 $8000 \rightarrow (8000-0)/50000 = 0.16$

活跃天数归一化（范围 0-365）：
 $30 \rightarrow (30-0)/365 = 0.08$

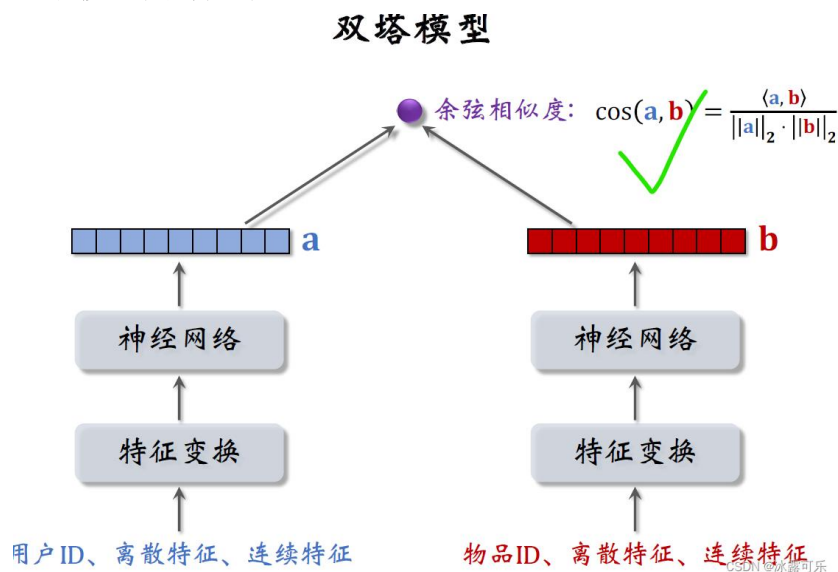
步骤2：拼接

连续特征向量 = [0.25, 0.16, 0.08]

做完特征处理，得到很多特征向量，把这些向量都拼起来输入神经网络。做召回用到这个向量



双塔模型图片如下：



双塔模型的训练

1. Pointwise：独立看待每个正样本、负样本，做简单的二元分类。
2. Pairwise：每次取一个正样本、一个负样本 [1]。
3. Listwise：每次取一个正样本、多个负样本 [2]。

参考文献：

1. Jui-Ting Huang et al. Embedding-based Retrieval in Facebook Search. In *KDD*, 2020.
2. Xinyang Yi et al. Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations. In *RecSys*, 2019.

CSDN @冰露可乐

pointwise 训练：数据集中包含正负样本，独立看待每个正样本和负样本，做简单的二元分类训练模型，在数据集上做随机梯度下降训练双塔模型。

Pointwise训练

- 把召回看做二元分类任务。
- 对于正样本，鼓励 $\cos(a, b)$ 接近 +1。
- 对于负样本，鼓励 $\cos(a, b)$ 接近 -1。
- 控制正负样本数量为 1:2 或者 1:3。

CSDN @冰露可乐

Parawise 训练：每次取一个正样本，一个负样本组成一个二元组，目标让正样本分数大于负样本分数（类似于 DPO 啊，所以 LOSS 也有 BT 理论）。损失函数用 triplely hing loss 或者 triple logistic loss.

$\text{Loss} = \max(0, 1 - (\text{正样本分数} - \text{负样本分数}))$ ，差值越大，Loss 越小。

$\text{Loss} = -\log(\sigma(\text{正样本分数} - \text{负样本分数}))$

Listwise 训练：每次取一个正样本和多个负样本组成一个 list。训练方式类似于多元分类，让正样本的概率最高。损失函数：Softmax Cross-Entropy Loss

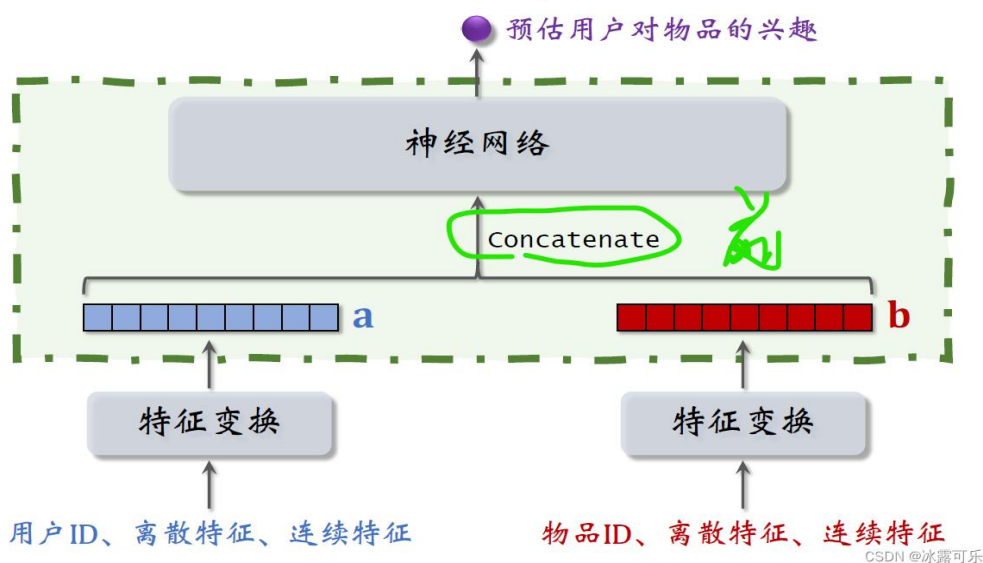
$\text{Loss} = -\log(\exp(\text{正样本分数}) / \sum \exp(\text{所有样本分数}))$

双塔模型

- 后期融合：把用户、物品特征分别输入不同的神经网络，不对用户、物品特征做融合。
- 线上计算量小：
 - 用户塔只需要做一次线上推理，计算用户表征 **a**。
 - 物品表征 **b** 事先储存在向量数据库中，物品塔在线上不做推理。
- 预估准确性不如精排模型。

CSDN @冰露可乐

不适用于召回的模型



上述模型用于排序，属于前期融合（在神经网络前就融合）。假如把这种精排的模型用于召回，就必须把所有物品的特征都挨个输入模型，预估用户对所有物品的兴趣。假设一共有 1

亿个物品，每给用户做一次召回，就要把这个模型跑 1 亿次，这种计算量显然不可行。

以后大家一看到这种前期融合模型，就要明白这是排序模型，不是召回模型。

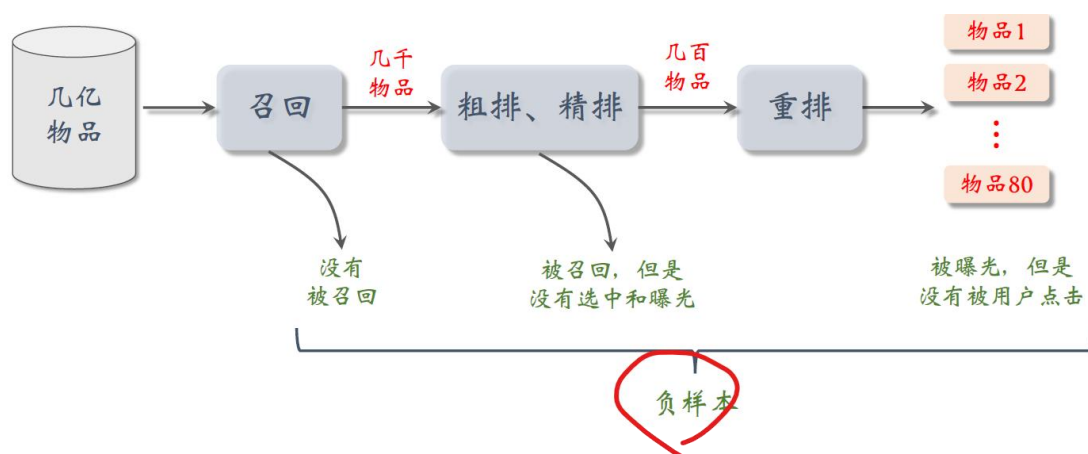
召回只能用双塔那样的后期融合模型。

那么该如何选择正负样本？

正样本很简单，就是用户点击过的物品，用户点击过一个物品，就说明用户对这个物品感兴趣。负样本的意思是用户不感兴趣的，负样本的选择有很多种。

正样本

- 正样本：曝光而且有点击的用户—物品二元组。
(用户对物品感兴趣)。
- 问题：少部分物品占据大部分点击，导致正样本大多是热门物品。
- 解决方案：过采样冷门物品，或降采样热门物品。
 - 过采样 (up-sampling)：一个样本出现多次。
 - 降采样 (down-sampling)：一些样本被抛弃。 CSDN @冰露可乐



但是这仨不能同时用于某一个阶段：比如曝光未点击的千万不能用于召回！（这部分不能当做召回模型的负样本，是用来训练排序模型的）

对于未被召回的物品：

方法一：

简单负样本：全体物品

- 未被召回的物品，大概率是用户不感兴趣的。
- 未被召回的物品 $\approx$ 全体物品 ✓
- 从全体物品中做抽样，作为负样本。

CSDN @冰露可乐

但此时的抽样要做到随机非均匀抽样，按热门程度加权（点击次数 $^{0.75}$ ），这样热门物品被抽为负样本的概率高，可以“打压”热门物品避免过度推荐，同时 0.75 次方平滑防止极端热门物品被过度打压，让冷门物品也有一定概率被抽到。这个经验值来自 Word2Vec 论文。

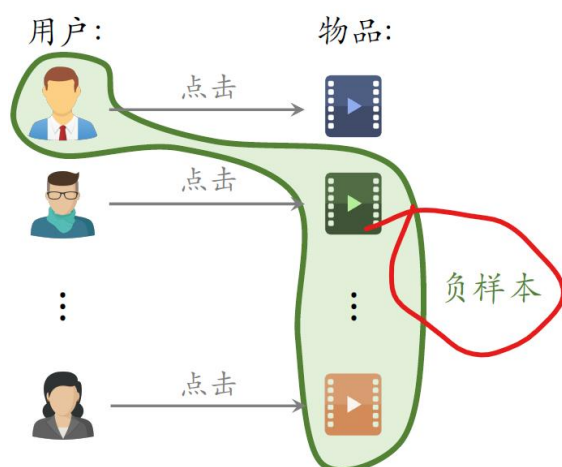
非均匀抽样：目的是打压热门物品

- 负样本抽样概率与热门程度（点击次数）正相关。
- 抽样概率 $\propto$ (点击次数) $^{0.75}$ 。

CSDN @冰露可乐

方法 2: batch 内负样本：

简单负样本：Batch内负样本



- 一个 batch 内有 n 个正样本。
- 一个用户和 $n-1$ 个物品组成负样本。
- 这个 batch 内一共有 $n(n-1)$ 个负样本。
- 都是简单负样本。（因为第一个用户不喜欢第二个物品。）

CSDN @冰露可乐

batch 内负样本存在一个问题，图中这些二元组都是通过点击行为选取的，第一

个用户和第一个物品之所以成为一个正样本，原因是用户点击了物品，所以一个物品出现在 **batch** 内的概率正比于它的点击次数，也就是它的热门程度，使得热门物品作为负样本的概率过大。

困难负样本：物品被召回，但是被粗排淘汰

困难负样本

- 困难负样本：
 - 被粗排淘汰的物品（比较困难）。
 - 精排分数靠后的物品（非常困难）。
- 对正负样本做二元分类：
 - 全体物品（简单）分类准确率高。
 - 被粗排淘汰的物品（比较困难）容易分错。
 - 精排分数靠后的物品（非常困难）更容易分错。

CSDN @冰露可乐

训练数据

- 混合几种负样本。
- 50%的负样本是全体物品（简单负样本）。
- 50%的负样本是没通过排序的物品（困难负样本）。

CSDN @冰露可乐

我解释一下召回选择负样本的原理：

召回的目的是快速找到用户可能感兴趣的物品，凡是用户可能感兴趣的全都取回来，然后再交给后面的排序模型逐一做甄别。召回模型的任务是区分用户不感兴趣的物品和可能感兴趣的物品，而不是区分比较感兴趣的物品和非常感兴趣的物品【这是排序干的事】，这就是选择负样本的基本思路。

总结

- ➡ • **正样本**：曝光而且有点击。
- ➡ • **简单负样本**：
 - 全体物品。
 - batch内负样本。
- ➡ • **困难负样本**：被召回，但是被排序淘汰。
- ➡ • **错误**：曝光、但是未点击的物品做召回的负样本。

CSDN @冰露可乐

双塔模型的召回如何部署：

将物品特征经过神经网络提取为向量后，在向量数据库中（MYSQL）存储为<特征向量, 物品 ID>的形式，便于后续进行最近邻查找。向量数据库会建立索引，建索引就是把向量空间划分成很多区域，每个区域用一个向量表示，这样可以加速最近邻查找。

用户向量不需要提前离线存储，当用户发起推荐请求的时候，调用神经网络在线上线算一个特征向量 **a**，然后把向量 **a** 作为 **query** 去数据库上做检索查找最近邻。

双塔模型缺点：

1. 训练目标和真实目标不一致

双塔通常训练的是：用户向量 **u** 和正样本 **item** 向量更近、用户向量 **u** 和负样本 **item** 向量更远。但真实业务目标可能是点击率、转化率、GMV、停留时长、满意度、长期留存，这两者并不完全等价。尤其是负采样会带来偏差：随机负样本 $\neq$ 用户真正不喜欢的 **item**、曝光未点击 $\neq$ 用户一定不感兴趣、batch 内负样本 $\neq$ 合理负样本。

2. 向量压缩带来信息瓶颈

3. 双塔缺乏深度交互

用户和物品在最后一步才交互。但很多推荐判断依赖细粒度交叉关系，比如：用户最近搜“轻薄电脑” $\times$ 商品标题是否包含“轻薄”、用户喜欢高端品牌 $\times$ 商品品牌、用户所在城市 $\times$ 商品配送时效，双塔很难充分建模这种交叉特征。

全量更新 vs 增量更新

全量更新：今天凌晨，用昨天全天的数据训练模型。

- 在昨天模型参数的基础上做训练。（不是随机初始化）
- 用昨天的数据，训练 1 epoch，即每天数据只用一遍。
- 发布新的用户塔神经网络和物品向量，供线上召回使用。
- 全量更新对数据流、系统的要求比较低。

CSDN @冰露可乐

全量更新 vs 增量更新

增量更新：做 online learning 更新模型参数。

- 用户兴趣会随时发生变化。
- 实时收集线上数据，做流式处理，生成 TFRecord 文件。
- 对模型做 online learning，增量更新 ID Embedding 参数。（不更新神经网络其他部分的参数。）
- 发布用户 ID Embedding，供用户塔在线上计算用户向量。

CSDN @冰露可乐

增量更新只更新 embedding 的参数，

更新模型

- **全量更新**：今天凌晨，用昨天的数据训练整个神经网络，做 1 epoch 的随机梯度下降。
- **增量更新**：用实时数据训练神经网络，只更新 ID Embedding，锁住全连接层。
- **实际的系统**：
 - **全量更新 & 增量更新** 相结合。
 - 每隔几十分钟，发布最新的用户 ID Embedding，供用户塔在线上计算用户向量。

CSDN @冰露可乐

召回通道：item cf、swing 双塔、地理位置召回、作者召回、缓存召回等等

精排：

排序的依据

- 排序模型预估点击率、点赞率、收藏率、转发率等多种分数。✓
- 融合这些预估分数。（比如加权和。）
- 根据融合的分数做排序、截断。

CSDN @冰露可乐

1. 多目标模型

输入是各种各样的特征



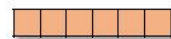
用户特征



物品特征



统计特征



场景特征

CSDN @冰露可乐

小结

1. 用户画像特征。
2. 笔记画像特征。
3. 用户统计特征。
4. 笔记统计特征。
5. 场景特征。

CSDN @冰露可乐

用户统计特征

- 用户最近30天（7天、1天、1小时）的曝光数、点击数、点赞数、收藏数……
- 按照笔记图文/视频分桶。（比如最近7天，该用户对图文笔记的点击率、对视频笔记的点击率。）
- 按照笔记类目分桶。（比如最近30天，用户对美妆笔记的点击率、对美食笔记的点击率、对科技数码笔记的点击率。）

CSDN @冰露可乐

特征处理

- 离散特征：做 embedding。
 - 用户ID、笔记ID、作者ID。
 - 类目、关键词、城市、手机品牌。
- 连续特征：做分桶，变成离散特征。
 - 年龄、笔记字数、视频长度。
- 连续特征：其他变换。
 - 曝光数、点击数、点赞数等数值做 $\log(1+x)$ 。
 - 转化为点击率、点赞率等值，并做平滑。

CSDN @冰露可乐

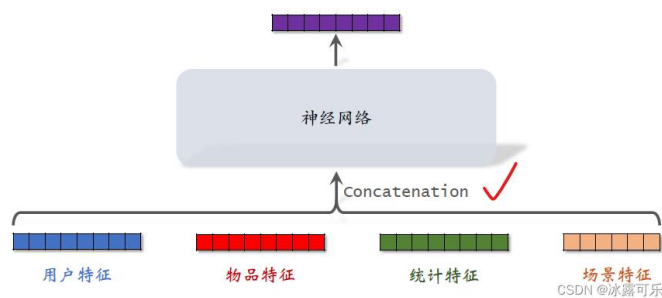
用户特征主要是用户 ID 和用户画像，

物品特征，包括物品 ID，物品画像，还有作者信息。

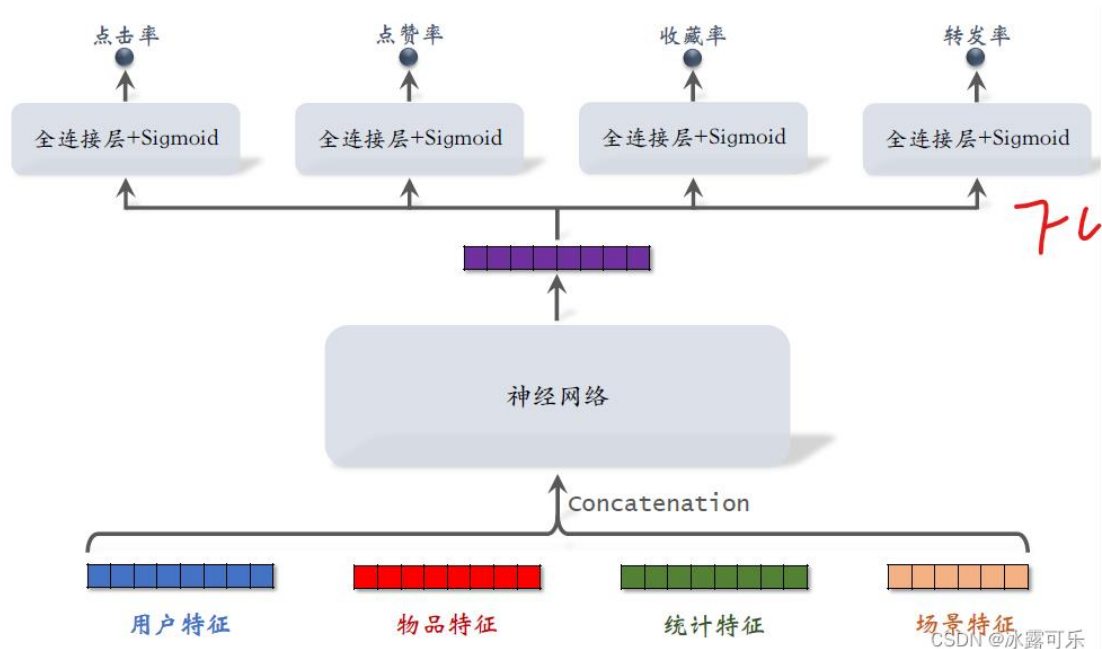
统计特征包括用户统计特征和物品统计特征。比如用户在过去 30 天中一共曝光了多少篇笔记，点击了多少篇笔记，点赞了多少篇笔记，再比如候选物品，在过去 30 天中一共获得了多少次曝光机会，获得多少次点击、点赞

场景特征是随着用户请求传过来的，包含当前的时间、用户所在的地点，这些信息对推荐很有用，比方说候选物品跟用户如果在同一个城市，那么用户对物品会有更高的兴趣。再比如，当前是否是周末节假日，也会影响用户的兴趣。

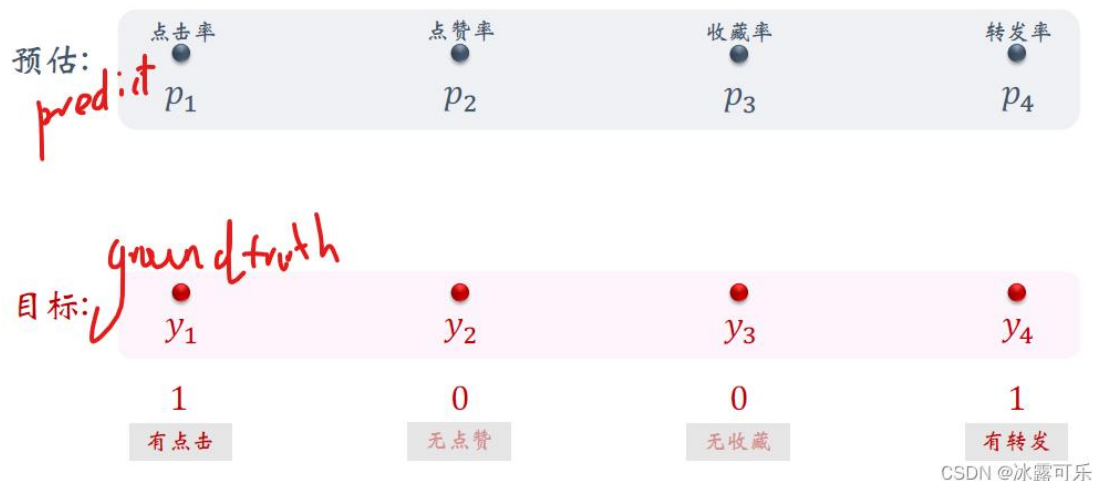
把这些特征做 concat 输入神经网络



神经网络会输出一个向量，这个向量再输入四个神经网络
四个神经网络分别输出点击率、点赞率、收藏率、转发率的预估值，四个预估值都是实数介于零到一之间。推荐系统排序就主要靠这四个预估值，他们反映出用户对物品的兴趣。



训练时，对于某个真实的物品，我们记录他的目标行为（0/1），让预测拟合。每个任务都是一个二元分类，我们就用交叉熵损失函数。把四个损失函数的加权和作为总的损失函数权重



实际训练的困难：样本不平衡问题

做训练的时候存在类别不平衡的问题，正样本少，负样本多，比方说每给用户曝光 100 篇笔记用户，点击十篇，其他 90 篇没有点击，有点击的是正样本，没有点击的是负样本。这负样本的数量非常不平衡，较多的负样本用途不大，白白浪费计算资源。

解决方案就是副样本的随机降采样 down-sampling。负样本过多，所以我们不用全部的负样本，只用其中一小部分负样本，这样的话负样本的数量会稍微平衡一些，同时可以减少样本数量，降低训练的计算代价。

但是在此之后需要校准。

设正样本和负样本的数量分别是 N_+ 和 N_- ，以点击为例，曝光之后有点击就是个正样本，否则就是负样本。负样本数量通常远大于正样本，在训练的时候会对负样本做降采样，抛弃一部分负样本，这样会让正负样本的差距不太悬殊。把采样率记作 α ，它介于零到一之间。

使用的负样本的数量等于阿尔法乘以 N_- ，由于减少了负样本数量，模型预估的点击率会大于真实点击率。阿尔法越小负，样本越少，模型对点击率的高估就会越严重。因此需要校准

- 真实点击率： $p_{\text{true}} = \frac{n_+}{n_+ + n_-}$ （期望）。

- 预估点击率： $p_{\text{pred}} = \frac{n_+}{n_+ + \alpha \cdot n_-}$ （期望）。

- 由上面两个等式可得校准公式[1]：

$$p_{\text{true}} = \frac{\alpha \cdot p_{\text{pred}}}{(1 - p_{\text{pred}}) + \alpha \cdot p_{\text{pred}}}$$

参考文献：

1. Xinran He et al. Practical lessons from predicting clicks on ads at Facebook. In the 8th International Workshop on Data Mining for Online Advertising.

CSDN @冰露可乐

在线上做排序的时候，首先要模型预估点击率， P_{predict} ，然后我们用这个公式做校准，最后拿校准之后的点击率作为排序的依据。

精排模型

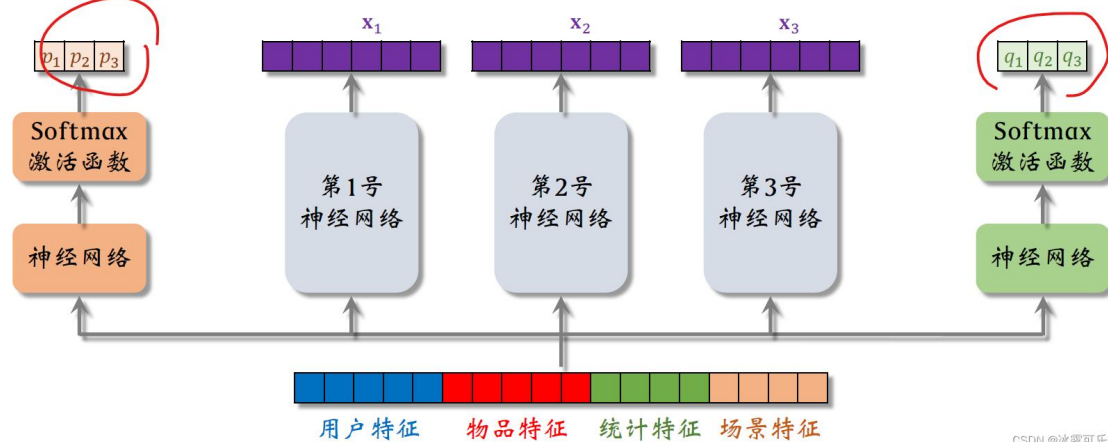
- 前期融合：先对所有特征做 concatenation，再输入神经网络。
- 线上推理代价大：如果有 n 篇候选笔记，整个大模型要做 n 次推理。

CSDN @冰露可乐

前期融合的意思是先对所有特征做 concatenation，然后再输入神经网络，这样的模型线上推理代价大。

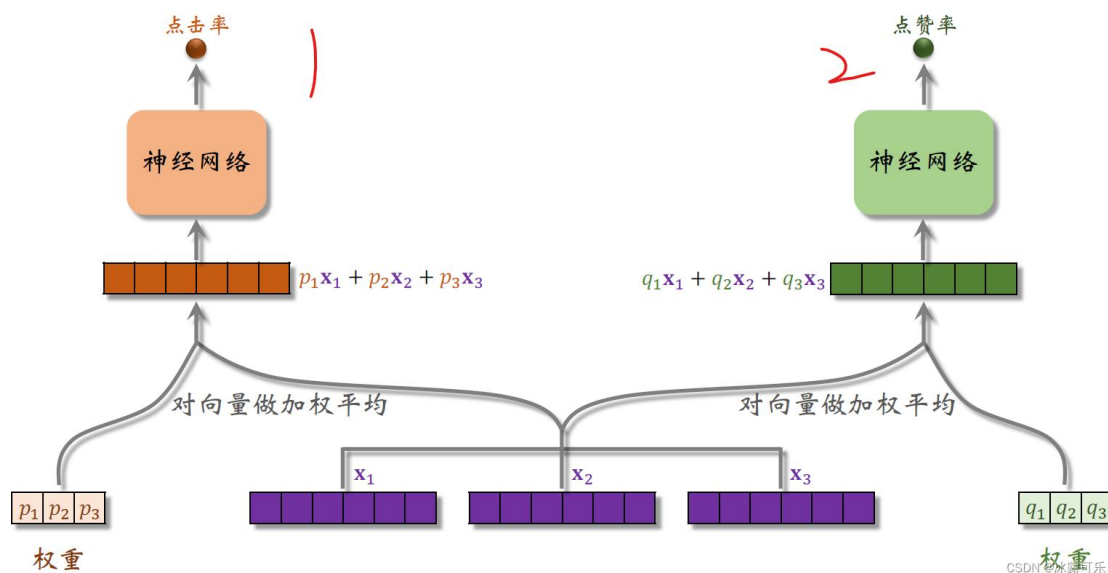
2. MMOE

第 1、2、3 号神经网络为专家，通常设置 4-8 个。两侧的神经网络为权重预测，这里我们假设 MMOE 只有两个目标点赞率与点击率，真实场景下有几个目标就有几个权重神经网络。



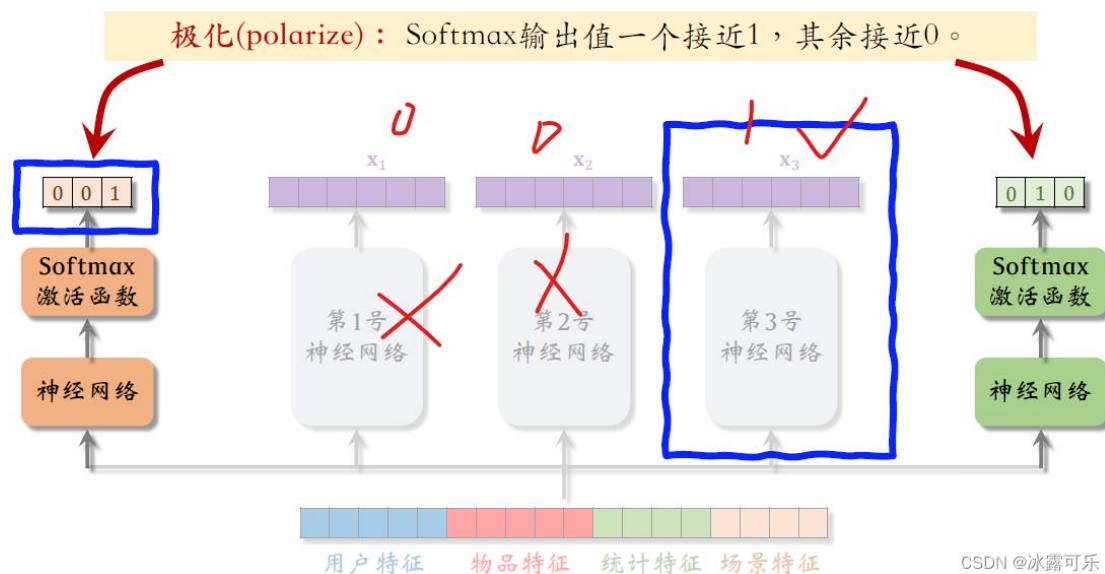
CSDN @冰露可乐

随后我们对专家输出的向量和权重进行加权平均，得到某一个预测目标的值：



CSDN @冰露可乐

问题：MMoE 实践过程中发现问题，被极化了，就是老是只有一个专家有效，权重是 001 的形式。也就是说，左边的预估点击率任务只使用了第三号专家神经网络，而没有使用其他两个专家神经网络



解决方案是在训练的过程对 soft max 输出使用 dropout：soft max 输出的 N 个数值被 mark 的概率都是 10%，也就是说在训练的过程中，每个专家被丢弃的概率都是 10%，这样会强迫每个任务根据部分专家做预测。

假如发生极化 soft max 输出的某个元素接近 1，这个元素被 mask 预测的结果肯定会错的离谱。为了让预测尽量精准，神经网络会尽量避免极化的发生，避免 soft max 输出的某个元素接近 1

解决极化问题

- 如果有 n 个“专家”，那么每个 softmax 的输入和输出都是 n 维向量。
- 在训练时，对 softmax 的输出使用 dropout。
 - Softmax 输出的 n 个数值被 mask 的概率都是 10%。
 - 每个“专家”被随机丢弃的概率都是 10%。

CSDN @冰露可乐

上面我们可以预估出多个分数，现在我们该怎么进行融合呢？

1. 求各种预估值值的加权和

融合预估分数

简单的加权和

$$p_{\text{click}} + w_1 \cdot p_{\text{like}} + w_2 \cdot p_{\text{collect}} + \dots$$

点击率乘以其他项的加权和

$$p_{\text{click}} \cdot (1 + w_1 \cdot p_{\text{like}} + w_2 \cdot p_{\text{collect}} + \dots)$$

CSDN @冰

主要分数依据，

图文 vs 视频

- 图文笔记排序的主要依据：

✓ 点击、点赞、收藏、转发、评论……

- 视频排序的依据还有播放时长和完播。
- 直接用回归拟合播放时长效果不好。建议用 YouTube 的时长建模 [1]。

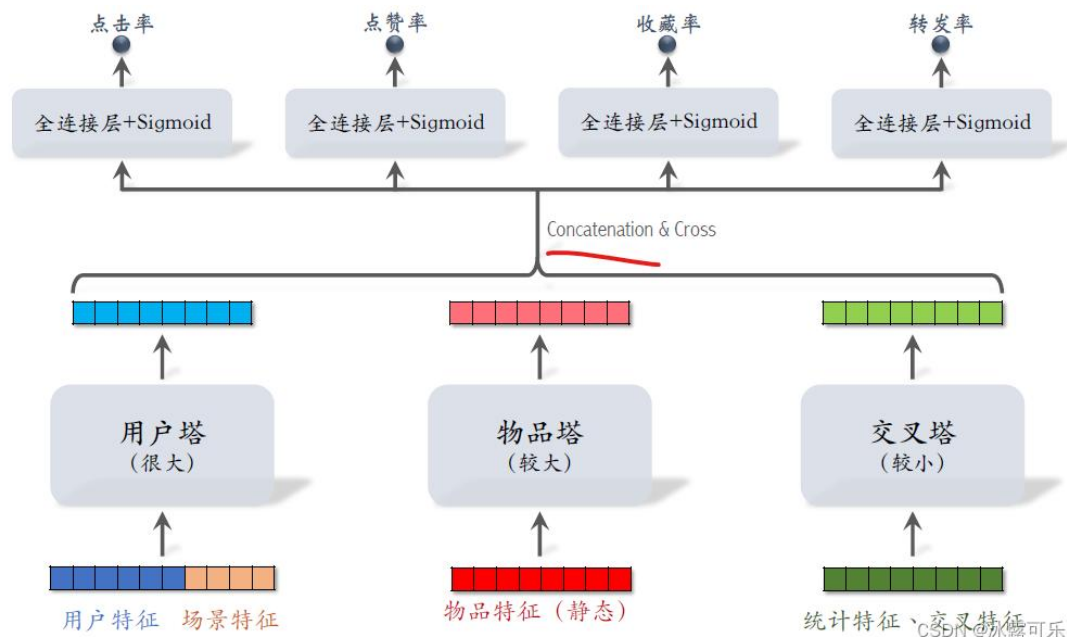
参考文献：

1. Paul Covington, Jay Adams, & Emre Sargin. Deep Neural Networks for YouTube Recommendations. In RecSys, 2016.

CSDN @冰露可乐

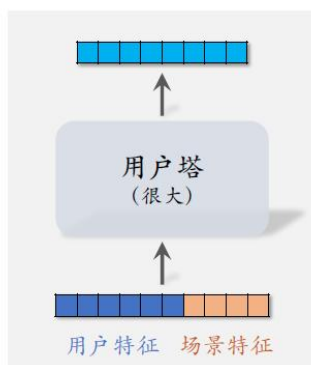
因此后期融合模型用于召回，而前期融合模型可以作为精排。

粗排：三塔模型

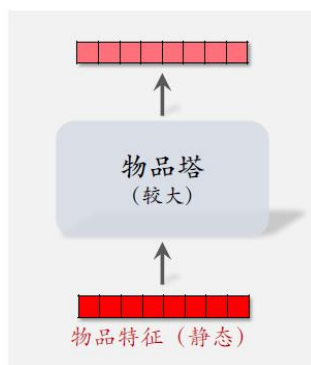


粗排的三塔模型

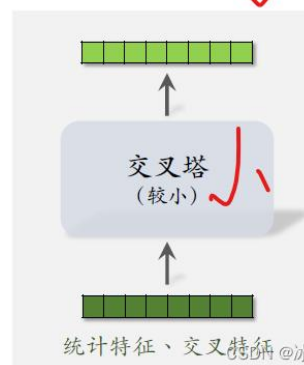
- 只有一个用户，用户塔只做一次推理。
- 即使用户塔很大，总计计算量也不大。



- 有 n 个物品，理论上物品塔需要做 n 次推理。
- PS 缓存物品塔的输出向量，避免绝大部分推理。



- 统计特征动态变化，缓存不可行。
- 有 n 个物品，交叉塔必须做 n 次推理。



用户塔：线上每次给一个用户做推荐，用户塔只需要做一次推理，即使用户塔很大，推理很慢也没有关系，用户塔对粗排总的计算量影响很小。

物品塔：物品可以存在缓存中，因为物品特性稳定。所以每次只需要对召回物品中的极少数无缓存物品进行计算，很快。

交叉塔：它的输入是用户和物品的统计特征，还有用户和物品特征的交叉。统计特征会实时动态变化，每当一个用户发生点击等行为，它的统计特征就会发生变化，每当一个物品获得曝光和交互，它的点击次数、点击率等指标就会发生变化。交叉塔在线上的推理避免不掉，粗排给 N 个物品打分，有 N 个物品的统计特征和交叉特征。交叉塔要实实在在做 N 次推理，所以交叉塔必须足够小，计算够快。通常来说交叉塔只有一层，宽度也比较小。

模型的上层结构：粗排给 N 维度打分，模型上层需要做 N 次推理，无法用缓存的方式，避

免不了 N 次计算。粗排推理的大部分计算量在模型上层。

三塔模型的推理

- 从多个数据源取特征：
 - 1 个用户的画像、统计特征。
 - n 个物品的画像、统计特征。
- 用户塔：只做 1 次推理。
- 物品塔：未命中缓存时需要做推理。
- 交叉塔：必须做 n 次推理。
- 上层网络做 n 次推理，给 n 个物品打分。

CSDN @冰露可乐

物品冷启动：

新笔记冷启动

为什么要特殊对待新笔记？

- 新笔记 缺少与用户的交互，导致推荐的难度大、效果差。
- 扶持新发布、低曝光的笔记，可以增强作者发布意愿。

CSDN @冰露可乐

优化冷启的目标

1. 精准推荐：克服冷启的困难，把新笔记推荐给合适的用户，不引起用户反感。
2. 激励发布：流量向低曝光新笔记倾斜，激励作者发布。
3. 挖掘高潜：通过初期小流量的试探，找到高质量的笔记，给与流量倾斜。

评价指标

→ • 作者侧指标：

- 发布渗透率、人均发布量。

→ • 用户侧指标：

- 新笔记指标：新笔记的点击率、交互率。
- 大盘指标：消费时长、日活、月活。

→ • 内容侧指标：

- 高热笔记占比。

CSDN @冰露可乐

作者侧指标：

渗透率等于当日发布人数除以日活人数。不论他这一天是发一篇，还是发十篇，他都只算一个发布人数。

这两个指标反映出作者的发布积极性。我们优化新笔记冷启动最重要的目标就是提高作者的积极性，促进发布

冷启召回的困难

- 缺少用户交互，还没学好笔记 ID embedding，导致双塔模型效果不好。
- 缺少用户交互，导致 ItemCF 不适用。

CSDN @冰露可乐

召回通道

✗ ItemCF召回 (不适用) ✗

❓ 双塔模型 (改造后适用)

✓ 类目、关键词召回 (适用)

✓ 聚类召回 (适用)

✓ Look-Alike召回 (适用)

CSDN @冰露可乐

改造双塔模型应对冷启动

ID Embedding

改进方案 1：新笔记使用 default embedding。

- 物品塔做 ID embedding 时，让所有新笔记共享一个 ID，而不是用自己真正的 ID。
- Default embedding：共享的 ID 对应的 embedding 向量。
- 到下次模型训练的时候，新笔记才有自己的 ID embedding 向量。

CSDN @冰露可乐

1. 让所有新笔记共享一个 **ID embedding**，而不是用新笔记自己真正的 ID。这个向量是学出来的，而不是随机初始化或者全 0 初始化的。比随机初始化的全 0 初始化更好。
2. 新笔记发布之后，查找 top k 内容最相似的高曝光笔记。在小红书的应用中相似，可以用图片文字类目来定义。用多模态神经网络把一篇笔记的图文内容表征为一个向量。每当一篇新笔记发布的时候，寻找最相似的 K 的向量就行，把找到的 K 篇高曝光笔记的 embedding 向量取平均作为新笔记的 embedding。之所以用高曝光笔记，是因为他们的 ID embedding 通常学的比较好，

用户冷启动和视频冷启动最大的区别在于：

用户冷启动缺少的是用户画像，系统不知道用户喜欢什么；

而视频冷启动缺少的是内容反馈数据，系统不知道这个视频质量如何。

因此两者解决思路不同：

用户冷启动重点是快速建立兴趣画像，比如利用用户标签、热门推荐和探索机制；

而视频冷启动重点是快速评估内容质量，比如通过内容特征、多模态 embedding，以及小流量测试机制来判断视频是否值得继续分发。

“用户冷启动和商品（内容）冷启动的核心区别在于：用户冷启动缺少的是用户行为数据，系统不知道用户喜欢什么；而商品冷启动缺少的是商品交互数据，系统不知道这个商品好不好、该不该推荐。因此，用户冷启动重点是快速建立用户画像，而商品冷启动重点是快速评估内容质量和分发价值。

“用户冷启动指的是新用户缺少历史行为数据，导致推荐系统无法准确建立用户画像的问题。

常见问题包括：推荐不精准、用户体验差、协同过滤失效、推荐过于热门化

常见解决方案有：

第一，热门推荐作为兜底；

第二，通过注册兴趣选择、人口属性等方式初始化用户画像；

第三，使用基于刚才看过内容的推荐，不依赖历史行为；

第四，通过 Explore（啥类型都推一点）或 Bandit 机制主动探索用户兴趣；

第五，在现代推荐系统中，也会利用 embedding 和多模态特征快速初始化用户向量。”

物品冷启动：

内容特征计算相似度，将新物品推荐给喜欢相似老物品的用户

多模态理解

小范围测试，收集反馈

训练超参数的选择逻辑。学习率 $3e-4$ +AdamW：自编码器类模型的标准配置。常数学习率调度+warmup：warmup 阶段让码本有时间通过 KMeans 初始化稳定下来，之后保持恒定学习率持续优化。梯度裁剪（clip_grad_norm=1.0）：量化操作的梯度直通估计可能产生较大的梯度，裁剪防止训练不稳定。batch_size=8192：向量量化对 batchsize 比较敏感，较大的 batch 能让 KMeans 初始化和 Sinkhorn 分配更准确。随机种子固定（seed=2025）：保证实验可复现